FastAPI Reference

Test Client - `TestClient`

You can use the `TestClient` class to test FastAPI applications without creating an actual HTTP and socket connection, just communicating directly with the FastAPI code.

Read more about it in the FastAPI docs for Testing.

You can import it directly from `fastapi.testclient`:

```
from fastapi.testclient import TestClient
```

class fastapi.testclient.TestClient

```
TestClient(
    app,
    base_url="http://testserver",
    raise_server_exceptions=True,
    root_path="",
    backend="asyncio",
    backend_options=None,
    cookies=None,
    headers=None,
    follow_redirects=True,
    client=("testclient", 50000),
)
```

Bases: `Client`

Ask AI

Source code in `starlette/testclient.py`

```
def __init__(
    self,
    app: ASGIApp,
    base_url: str = "http://testserver",
    raise_server_exceptions: bool = True,
    root_path: str = "",
    backend: Literal["asyncio", "trio"] = "asyncio",
    backend_options: dict[str, Any] | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    headers: dict[str, str] | None = None,
    follow_redirects: bool = True,
    client: tuple[str, int] = ("testclient", 50000),
) -> None:
    self.async_backend = _AsyncBackend(backend=backend, backend_options=backend_options or {})
    if _is_asgi3(app):
        asgi_app = app
    else:
        app = cast(ASGI2App, app)  # type: ignore[assignment]
        asgi_app = _WrapASGI2(app)  # type: ignore[arg-type]
    self.app = asgi_app
    self.app_state: dict[str, Any] = {}
    transport = _TestClientTransport(
        self.app,
        portal_factory=self._portal_factory,
        raise_server_exceptions=raise_server_exceptions,
        root_path=root_path,
        app_state=self.app_state,
        client=client,
    )
    if headers is None:
        headers = {}
    headers.setdefault("user-agent", "testclient")
    super().__init__(
        base_url=base_url,
        headers=headers,
        transport=transport,
        follow_redirects=follow_redirects,
        cookies=cookies,
    )
```

`attr` headers `property` `writable`

```
headers
```

HTTP headers to include when sending requests.

`attr` follow_redirects `instance-attribute`

```
follow_redirects = follow_redirects
```

`attr` max_redirects `instance-attribute`

```
max_redirects = max_redirects
```

`attr` is_closed `property`

```
is_closed
```

Check if the client being closed

attr trust_env `property`

```
trust_env
```

attr timeout `property` `writable`

```
timeout
```

attr event_hooks `property` `writable`

```
event_hooks
```

attr auth `property` `writable`

```
auth
```

Authentication class used when none is passed at the request-level.

See also Authentication.

attr base_url `property` `writable`

```
base_url
```

Base URL to use when sending requests with relative URLs.

attr cookies `property` `writable`

```
cookies
```

Cookie values to include when sending requests.

attr params `property` `writable`

```
params
```

Query parameters to include in the URL when sending requests.

attr task `instance-attribute`

```
task
```

attr portal `class-attribute` `instance-attribute`

```
portal = None
```

attr async_backend `instance-attribute`

```
async_backend = _AsyncBackend(
    backend=backend, backend_options=backend_options or {}
```

```
)
```

attr app `instance-attribute`

```
app = asgi_app
```

attr app_state `instance-attribute`

```
app_state = {}
```

meth build_request

```
build_request(
    method,
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Build and return a request instance.

- The `params`, `headers` and `cookies` arguments are merged with any values set on the client.
- The `url` argument is merged with any `base_url` set on the client.

See also: Request instances

Source code in `httpx/_client.py`

```python
def build_request(
    self,
    method: str,
    url: URL | str,
    *,
    content: RequestContent | None = None,
    data: RequestData | None = None,
    files: RequestFiles | None = None,
    json: typing.Any | None = None,
    params: QueryParamTypes | None = None,
    headers: HeaderTypes | None = None,
    cookies: CookieTypes | None = None,
    timeout: TimeoutTypes | UseClientDefault = USE_CLIENT_DEFAULT,
    extensions: RequestExtensions | None = None,
) -> Request:
    """
    Build and return a request instance.

    * The `params`, `headers` and `cookies` arguments
    are merged with any values set on the client.
    * The `url` argument is merged with any `base_url` set on the client.

    See also: [Request instances][0]

    [0]: /advanced/clients/#request-instances
    """
    url = self._merge_url(url)
    headers = self._merge_headers(headers)
    cookies = self._merge_cookies(cookies)
    params = self._merge_queryparams(params)
    extensions = {} if extensions is None else extensions
    if "timeout" not in extensions:
        timeout = (
            self.timeout
            if isinstance(timeout, UseClientDefault)
            else Timeout(timeout)
        )
        extensions = dict(**extensions, timeout=timeout.as_dict())
    return Request(
        method,
        url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        extensions=extensions,
    )
```

`meth` stream

```python
stream(
    method,
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
```

```
)
```

Alternative to `httpx.request()` that streams the response body instead of loading it into memory at once.

Parameters: See `httpx.request`.

See also: Streaming Responses

Source code in `httpx/_client.py`

```
@contextmanager
def stream(
    self,
    method: str,
    url: URL | str,
    *,
    content: RequestContent | None = None,
    data: RequestData | None = None,
    files: RequestFiles | None = None,
    json: typing.Any | None = None,
    params: QueryParamTypes | None = None,
    headers: HeaderTypes | None = None,
    cookies: CookieTypes | None = None,
    auth: AuthTypes | UseClientDefault | None = USE_CLIENT_DEFAULT,
    follow_redirects: bool | UseClientDefault = USE_CLIENT_DEFAULT,
    timeout: TimeoutTypes | UseClientDefault = USE_CLIENT_DEFAULT,
    extensions: RequestExtensions | None = None,
) -> typing.Iterator[Response]:
    """
    Alternative to `httpx.request()` that streams the response body
    instead of loading it into memory at once.

    **Parameters**: See `httpx.request`.

    See also: [Streaming Responses][0]

    [0]: /quickstart#streaming-responses
    """
    request = self.build_request(
        method=method,
        url=url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        timeout=timeout,
        extensions=extensions,
    )
    response = self.send(
        request=request,
        auth=auth,
        follow_redirects=follow_redirects,
        stream=True,
    )
    try:
        yield response
    finally:
        response.close()
```

`meth` send

```
send(
    request,
    *,
    stream=False,
    auth=USE_CLIENT_DEFAULT,
```

```
    follow_redirects=USE_CLIENT_DEFAULT
)
```

Send a request.

The request is sent as-is, unmodified.

Typically you'll want to build one with `Client.build_request()` so that any client-level configuration is merged into the request, but passing an explicit `httpx.Request()` is supported as well.

See also: Request instances

Source code in `httpx/_client.py`

```
def send(
    self,
    request: Request,
    *,
    stream: bool = False,
    auth: AuthTypes | UseClientDefault | None = USE_CLIENT_DEFAULT,
    follow_redirects: bool | UseClientDefault = USE_CLIENT_DEFAULT,
) -> Response:
    """
    Send a request.

    The request is sent as-is, unmodified.

    Typically you'll want to build one with `Client.build_request()`
    so that any client-level configuration is merged into the request,
    but passing an explicit `httpx.Request()` is supported as well.

    See also: [Request instances][0]

    [0]: /advanced/clients/#request-instances
    """
    if self._state == ClientState.CLOSED:
        raise RuntimeError("Cannot send a request, as the client has been closed.")

    self._state = ClientState.OPENED
    follow_redirects = (
        self.follow_redirects
        if isinstance(follow_redirects, UseClientDefault)
        else follow_redirects
    )

    self._set_timeout(request)

    auth = self._build_request_auth(request, auth)

    response = self._send_handling_auth(
        request,
        auth=auth,
        follow_redirects=follow_redirects,
        history=[],
    )
    try:
        if not stream:
            response.read()

        return response

    except BaseException as exc:
        response.close()
        raise exc
```

meth close

```
close()
```

Close transport and proxies.

Source code in `httpx/_client.py`

```
def close(self) -> None:
    """
    Close transport and proxies.
    """
    if self._state != ClientState.CLOSED:
        self._state = ClientState.CLOSED

        self._transport.close()
        for transport in self._mounts.values():
            if transport is not None:
                transport.close()
```

`meth` request

```
request(
    method,
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def request(  # type: ignore[override]
    self,
    method: str,
    url: httpx._types.URLTypes,
    *,
    content: httpx._types.RequestContent | None = None,
    data: _RequestData | None = None,
    files: httpx._types.RequestFiles | None = None,
    json: Any = None,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    if timeout is not httpx.USE_CLIENT_DEFAULT:
        warnings.warn(
            "You should not use the 'timeout' argument with the TestClient. "
            "See https://github.com/Kludex/starlette/issues/1108 for more information.",
            DeprecationWarning,
        )
    url = self._merge_url(url)
    return super().request(
        method,
        url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

`meth` get

```
get(
    url,
    *,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def get(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().get(
        url,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

`meth` options

```
options(
    url,
    *,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def options(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().options(
        url,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

meth head

```
head(
    url,
    *,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def head(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
 httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
 httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().head(
        url,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

meth post

```
post(
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def post(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    content: httpx._types.RequestContent | None = None,
    data: _RequestData | None = None,
    files: httpx._types.RequestFiles | None = None,
    json: Any = None,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().post(
        url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

`meth` put

```
put(
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def put(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    content: httpx._types.RequestContent | None = None,
    data: _RequestData | None = None,
    files: httpx._types.RequestFiles | None = None,
    json: Any = None,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().put(
        url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

`meth` patch

```
patch(
    url,
    *,
    content=None,
    data=None,
    files=None,
    json=None,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def patch(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    content: httpx._types.RequestContent | None = None,
    data: _RequestData | None = None,
    files: httpx._types.RequestFiles | None = None,
    json: Any = None,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().patch(
        url,
        content=content,
        data=data,
        files=files,
        json=json,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

`meth` delete

```
delete(
    url,
    *,
    params=None,
    headers=None,
    cookies=None,
    auth=USE_CLIENT_DEFAULT,
    follow_redirects=USE_CLIENT_DEFAULT,
    timeout=USE_CLIENT_DEFAULT,
    extensions=None
)
```

Source code in `starlette/testclient.py`

```
def delete(  # type: ignore[override]
    self,
    url: httpx._types.URLTypes,
    *,
    params: httpx._types.QueryParamTypes | None = None,
    headers: httpx._types.HeaderTypes | None = None,
    cookies: httpx._types.CookieTypes | None = None,
    auth: httpx._types.AuthTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    follow_redirects: bool | httpx._client.UseClientDefault = httpx._client.USE_CLIENT_DEFAULT,
    timeout: httpx._types.TimeoutTypes | httpx._client.UseClientDefault =
httpx._client.USE_CLIENT_DEFAULT,
    extensions: dict[str, Any] | None = None,
) -> httpx.Response:
    return super().delete(
        url,
        params=params,
        headers=headers,
        cookies=cookies,
        auth=auth,
        follow_redirects=follow_redirects,
        timeout=timeout,
        extensions=extensions,
    )
```

meth websocket_connect

```
websocket_connect(url, subprotocols=None, **kwargs)
```

Source code in `starlette/testclient.py`

```
def websocket_connect(
    self,
    url: str,
    subprotocols: Sequence[str] | None = None,
    **kwargs: Any,
) -> WebSocketTestSession:
    url = urljoin("ws://testserver", url)
    headers = kwargs.get("headers", {})
    headers.setdefault("connection", "upgrade")
    headers.setdefault("sec-websocket-key", "testserver==")
    headers.setdefault("sec-websocket-version", "13")
    if subprotocols is not None:
        headers.setdefault("sec-websocket-protocol", ", ".join(subprotocols))
    kwargs["headers"] = headers
    try:
        super().request("GET", url, **kwargs)
    except _Upgrade as exc:
        session = exc.session
    else:
        raise RuntimeError("Expected WebSocket upgrade")  # pragma: no cover

    return session
```

meth lifespan `async`

```
lifespan()
```

Source code in `starlette/testclient.py`

```
async def lifespan(self) -> None:
    scope = {"type": "lifespan", "state": self.app_state}
    try:
        await self.app(scope, self.stream_receive.receive, self.stream_send.send)
    finally:
        await self.stream_send.send(None)
```

meth wait_startup `async`

```
wait_startup()
```

Source code in `starlette/testclient.py`

```
async def wait_startup(self) -> None:
    await self.stream_receive.send({"type": "lifespan.startup"})

    async def receive() -> Any:
        message = await self.stream_send.receive()
        if message is None:
            self.task.result()
        return message

    message = await receive()
    assert message["type"] in (
        "lifespan.startup.complete",
        "lifespan.startup.failed",
    )
    if message["type"] == "lifespan.startup.failed":
        await receive()
```

meth wait_shutdown `async`

```
wait_shutdown()
```

Source code in `starlette/testclient.py`

```
async def wait_shutdown(self) -> None:
    async def receive() -> Any:
        message = await self.stream_send.receive()
        if message is None:
            self.task.result()
        return message

    await self.stream_receive.send({"type": "lifespan.shutdown"})
    message = await receive()
    assert message["type"] in (
        "lifespan.shutdown.complete",
        "lifespan.shutdown.failed",
    )
    if message["type"] == "lifespan.shutdown.failed":
        await receive()
```